HIGH PERFORMANCE COMPUTING

Assignment -5

Name : Juluri Pavan

HT NO : 2303A51412
Batch No : 21

Section 1: Serial Recursive Fibonacci (Baseline) Aim Measure the execution time of a serial recursive Fibonacci algorithm and understand the computational cost of recursion and overlapping subproblems. Requirements C compiler (gcc) Standard C libraries Linux / HPC lab system **Code :**

```
%%writefile fibonacci.c
#include <stdio.h>
#include <time.h>

long long fib(int n) { if (n <= 1)
return n; return fib(n - 1) +
fib(n - 2);
}

int main() {      int n;
printf("Enter n: ");
scanf("%d", &n);

    clock_t start = clock();
long long result = fib(n); clock_t end = clock(); double

time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("Fib(%d)     =     %lld\n",     n,     result);
    printf("Execution Time = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT :

```
!gcc fibonacci.c -o fibonacci!echo
40 | ./fibonacci

Overwriting fibonacci.c
```

Section 2: OpenMP Task-based Fibonacci (Basic Tasking) Aim Implement OpenMP taskbased parallelism for a recursive Fibonacci algorithm and observe performance

improvement over the serial version. Requirements C compiler with OpenMP support (gcc - fopenmp) OpenMP library **Code :**

```
%%writefile fibonacci_omp.c
#include <stdio.h>
#include <omp.h>

long long fib(int n) {
if (n <= 1) return n;
long long x, y;
    #pragma omp task shared(x)
x = fib(n - 1);
    #pragma omp task shared(y)
y = fib(n - 2);   #pragma
omp taskwait
    return x + y;
}

int main() {      int n;
printf("Enter n: "); scanf("%d",
&n); double start =
omp_get_wtime();

    long long result;
#pragma omp parallel
    {
        #pragma omp single
result = fib(n);
    } double end =

    omp_get_wtime();

    printf("Fib(%d)     =     %lld\n",     n,     result);
printf("Execution  Time  =  %f  seconds\n",  end  -  start);
return 0;}
```

OUTPUT :

```
!gcc -fopenmp fibonacci_omp.c -o fibonacci_omp
!echo 40 | ./fibonacci_omp Writing
```

fibonacci_omp.c

Section 3: Task Creation and Synchronization using taskwait Aim Understand task synchronization in OpenMP by using

#pragma omp taskwait to control execution order in recursive algorithms. Requirements

C compiler with OpenMP support OpenMP tasking directives

Code :

```
%%writefile fibonacci_omp_cutoff.c
#include <stdio.h>
#include <omp.h>

long long fib(int n) { if (n <= 20) {     //
    cutoff to reduce overhead
if (n <= 1) return n; return fib(n - 1)
        + fib(n - 2);
    } long long x,

    y;

    #pragma omp task shared(x)
x = fib(n - 1);

    #pragma omp task shared(y)
y = fib(n - 2);

    #pragma omp taskwait // synchronization point return
x + y;
}

int main() {      int n;
printf("Enter n: "); scanf("%d",
&n); double start =
omp_get_wtime();

    long long result;
#pragma omp parallel
    { // A single master thread creates the initial task
        #pragma omp single
result = fib(n);
    } double end = omp_get_wtime();

    printf("Fib(%d) = %lld\n", n,

    result); printf("Execution Time = %f
```

```
    seconds\n", end - start); return 0;}
```

OUTPUT :

```
!gcc -fopenmp fibonacci_omp_cutoff.c -o fibonacci_omp_cutoff!echo
40 | ./fibonacci_omp_cutoff
```

```
Writing fibonacci_omp_cutoff.c
```

Section 4: Performance Analysis of OpenMP Tasking Aim Analyze the performance overhead of OpenMP task creation by comparing execution time for different Fibonacci input sizes. Requirements C compiler with OpenMP Timing functions (omp_get_wtime)

Code :

```
%%writefile fibonacci_performance.c
#include <stdio.h>
#include <omp.h>

long long fib(int n) {     if
(n <= 20) {         if (n <=
1) return n; return fib(n - 1) + fib(n
        - 2);
    } long long x,

    y;

    #pragma omp task shared(x)
x = fib(n - 1);

    #pragma omp task shared(y)
y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() { for (int n = 30; n <= 45;
    n += 5) {

        double start = omp_get_wtime();

        long long result;
#pragma omp parallel
        {
            #pragma omp single
```

```
result = fib(n); } double end =
        omp_get_wtime();

        printf("n=%d Fib=%lld Time=%f sec\n", n, result, end - start);
    }
return 0;}
```

OUTPUT :

```
!gcc -fopenmp fibonacci_performance.c -o fibonacci_performance
!./fibonacci_performance
```

```
Writing fibonacci_performance.c
```

Section 5: Recursive Divide-and-Conquer Sum using OpenMP Tasks Aim Apply OpenMP tasking to a divide-and-conquer recursive array summation problem and compare it with loop-based parallelism. Requirements
 C compiler with OpenMP
 OpenMP task and taskwait directives **Code :**

```
%%writefile recursive_sum_omp.c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define SIZE 1000000
#define THRESHOLD 10000

long long sum(int *arr, int start, int end) {
if (end - start <= THRESHOLD) {     long long
s = 0; for (int i = start; i < end; i++)
s += arr[i];         return s;
    }

    int mid = (start + end) / 2;
long long s1, s2;

    #pragma omp task shared(s1)
    s1 = sum(arr, start, mid);

    #pragma omp task shared(s2)
    s2 = sum(arr, mid, end);
    #pragma omp taskwait
    return s1 + s2;
}
```

```
int main() {      int *arr =
malloc(sizeof(int) * SIZE);   for (int i
= 0; i < SIZE; i++)         arr[i] = 1;

    double start = omp_get_wtime();

    long long result;
#pragma omp parallel
    {
        #pragma omp single
result = sum(arr, 0, SIZE);
    } double end =

    omp_get_wtime();

    printf("Total Sum = %lld\n", result);     printf("Execution Time = %f
seconds\n", end - start);

    free(arr);
return 0;}
```

OUTPUT

:

```
!gcc -fopenmp recursive_sum_omp.c -o recursive_sum_omp
!./recursive_sum_omp
```

```
Writing recursive_sum_omp.c
```